



DEMONSTRATION, TECHNICAL PAPER AND WHITE PAPER IN ONE DECK

AquaZeroFit for Android

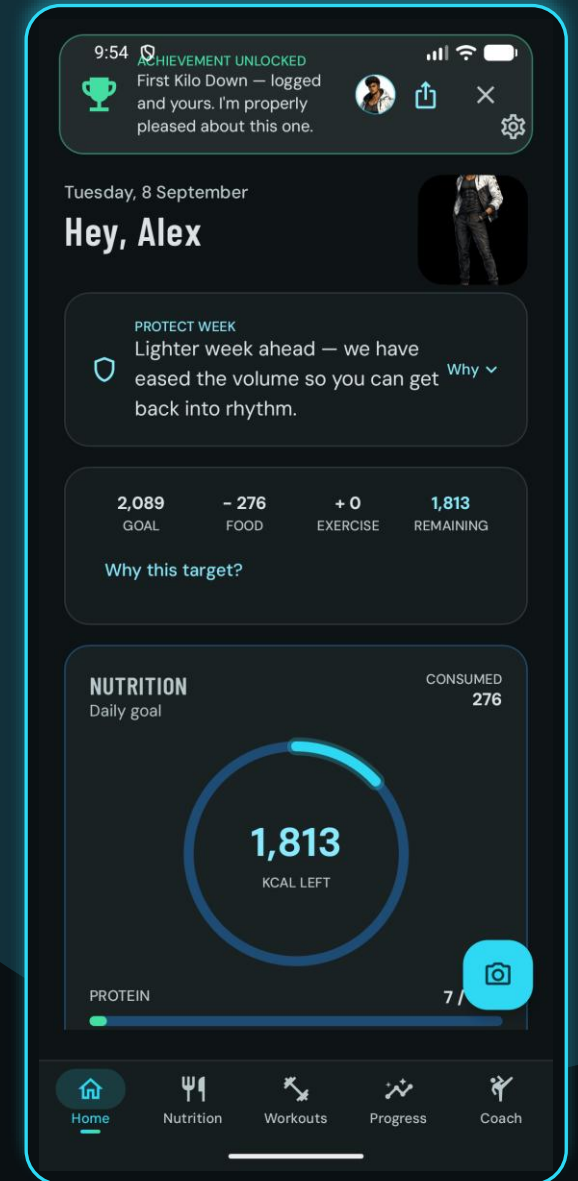
A native Kotlin and Jetpack Compose client for Google Play, built offline-first around one rule: models identify, code calculates.

Babatundji Williams-Fulwood

Student ID s8138393

NIT3213 Mobile Application Development, Victoria University, Melbourne [1]

App demonstration presentation, September 2026



Home tab, captured on a Pixel emulator running the debug build

AGENDA

Table of contents

01	Executive summary	3	13	Technical architecture (section)	15
02	The problem with fitness apps today	4	14	System context	16
03	Product tour: five tabs	5	15	Android technology stack	17
04	Photograph a meal, confirm first	6	16	Layered architecture	18
05	Four ways to log, plus barcodes	7	17	Unidirectional data flow	19
06	Targets you can check yourself	8	18	Offline-first outbox	20
07	Training: readiness and guided sessions	9	19	Networking, tokens and the Keystore vault	21
08	Exercise library with licence attribution	10	20	Dependency injection with Hilt	22
09	Aqua Coach and the persona roster	11	21	Native capabilities	23
10	Progress and the adaptive engine	12	22	Backend and AI gateway	24 to 25
11	Motivation that never punishes	13	23	Security, privacy and permissions	26
12	Why it beats the field	14	24	Quality, release and Google Play	27 to 29
			25	How it was built, outcomes, limits	30 to 32
			26	Conclusion, references, glossary	33 to 36

Slides 3 to 14 are written for every reader. Slides 15 to 29 go deeper for developers. Citations appear as bracketed numbers and resolve on the reference slides.

One app, one rule: models identify, code calculates

AquaZeroFit is an AI-assisted wellness app under the AquaZero brand [2]. The Android client is a separate native application, not a wrapper around the website, written in Kotlin with Jetpack Compose and shipped through Google Play [3][4]. A user photographs a meal, confirms what the model saw, and every calorie is then multiplied by code from a reference food corpus. The language model never writes a number the user will act on [5].

21

screens in Compose

50k

lines of production Kotlin

573

Android tests (518 JVM, 55 on device)

5.9 MB

signed release APK after R8



Offline-first

Room is the source of truth. Logs replay through an outbox with idempotency keys, so a gym with no signal still counts.



Safety in code

Calorie floors, allergen exclusion and crisis signposting are enforced by deterministic code, never by a model.



Honest numbers

Every target has a Why button that shows the formula. Attribution stays on every exercise record.



Nutrition tab

Workouts tab

The four failures we designed against



Silent wrong numbers

Photo loggers that write a calorie figure straight into the diary teach users to distrust the diary. AquaZeroFit never logs a photo result until the user confirms each item and portion.



AI that invents

A language model asked for calories will produce one whether or not it knows. Here the model returns identifiers and quantities only; kcal is grams times the per-100 g value from the corpus [5].



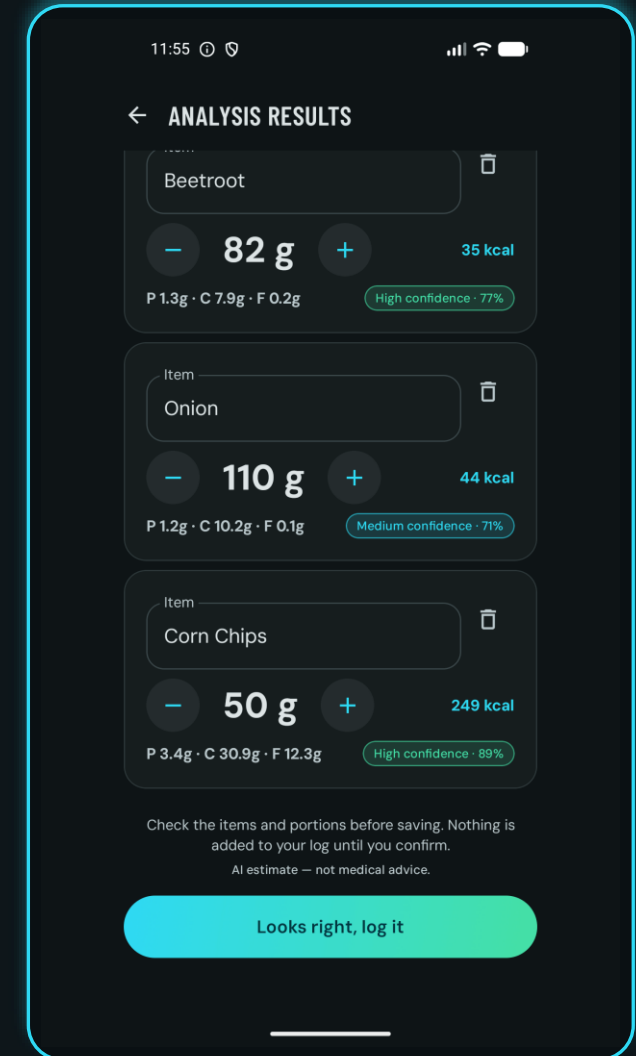
No signal in the gym

Basements and planes are where logging happens. Every log lands in Room first and syncs later with an Idempotency-Key, so nothing double-counts [4][10].



Medical creep

Wellness apps drift into diagnosis. The coach declines medical, crisis and extreme-diet requests in code and signposts real help, and the disclaimer is pinned on screen [5].



Confirm-first: nothing is added until the user says so

Five tabs, one daily loop



Home

Today's rings, readiness and the coach's line

Nutrition

Calories, macros, hydration and the meal timeline

Workouts

This week's plan, readiness and the library

Progress

Consistency, weight journey and calorie trend

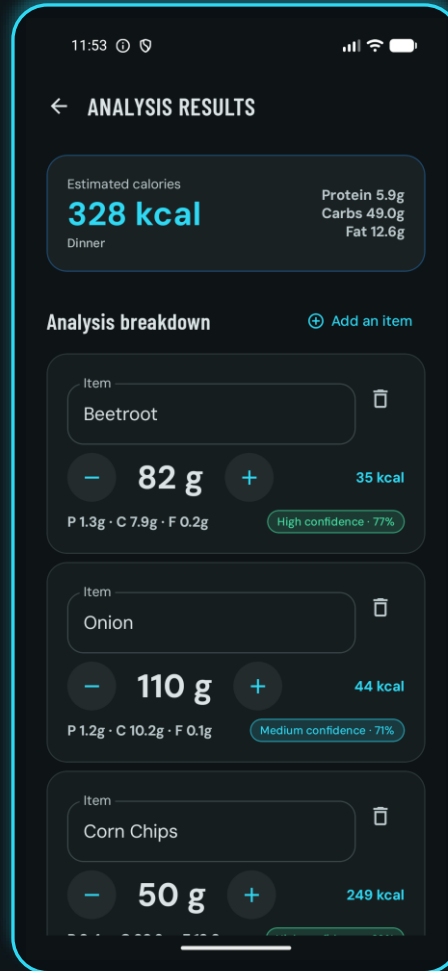
Coach

Akin, grounded in today's real data

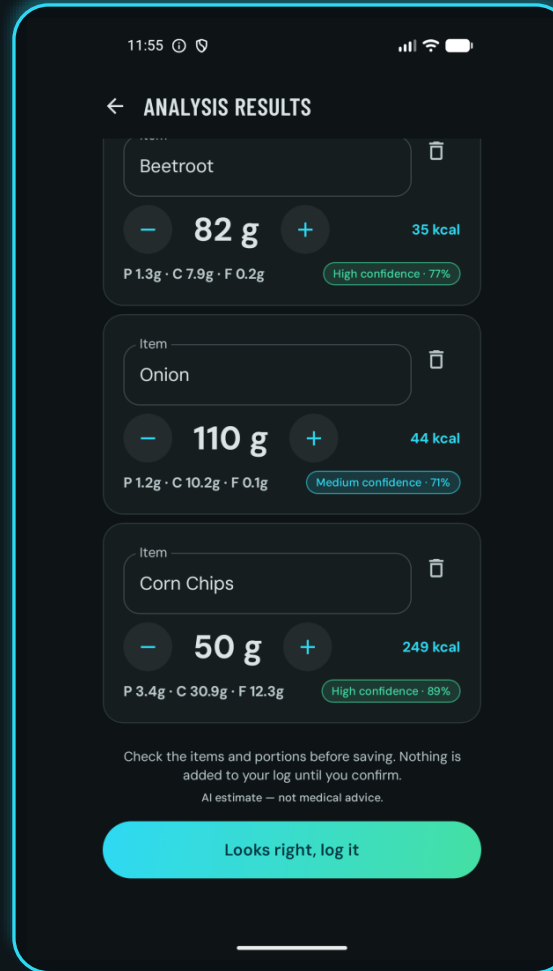
Photograph a meal, see it counted, confirm before it lands



1. CameraX capture



2. Items, portions, confidence



3. Edit, then confirm

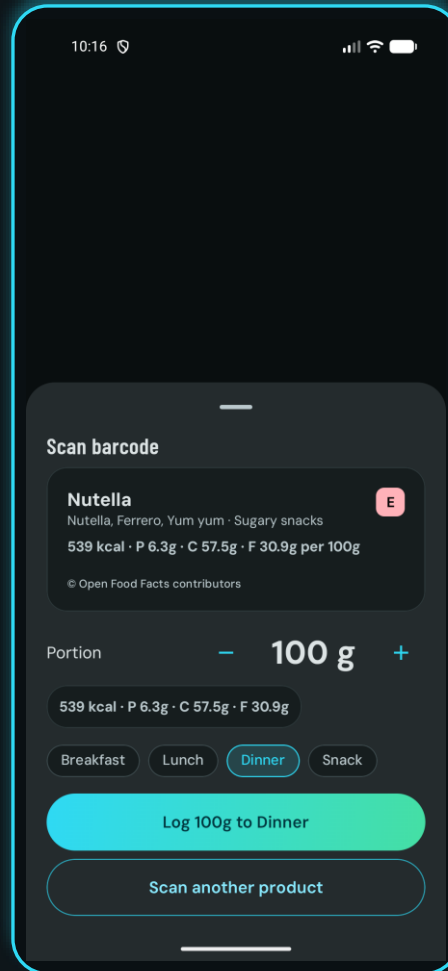
How it works

- The photo is stripped of EXIF, XMP and location metadata before upload and deleted from the server on confirm or after 24 hours [5].
- The vision model returns food identifiers, grams and a confidence score. It has no field for calories.
- Code multiplies grams by the per-100 g values in the food corpus, so 82 g of beetroot is always 35 kcal.
- Confidence bands (high, medium, low) come from a pure Kotlin function, tested on the JVM.
- A fat caution appears when the item names suggest frying, sauces or dressings.

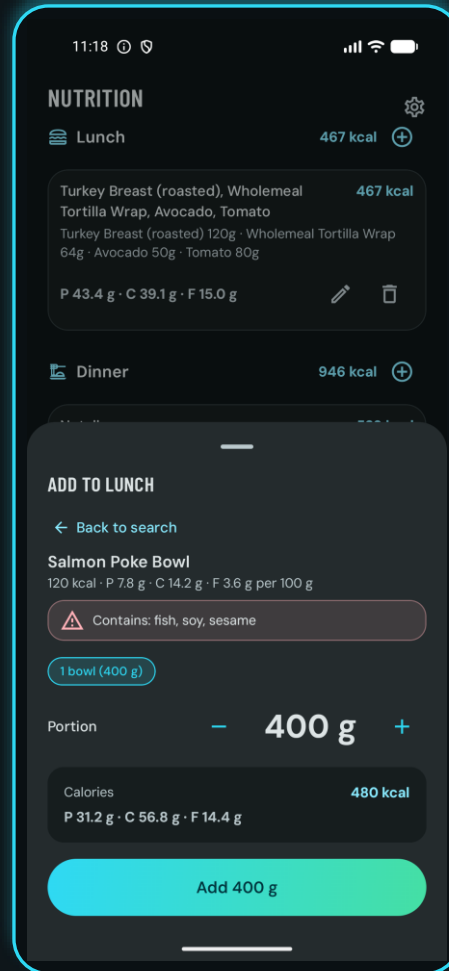
Four ways to log, and every one of them is free



Four ways to log



Barcode result with Open Food Facts credit



Search with allergen warning



ML Kit barcode scanning

On-device barcode detection through Google ML Kit, then a lookup against Open Food Facts data held in its own container with the ODbL credit preserved [13][21].



Searchable food corpus

139 curated foods with per-100 g macros and household portions. Recently used items are served from Room when offline.



Copy yesterday

One tap duplicates a previous day's meals into today, the fastest path for people who eat the same breakfast.



Hydration in one tap

Log 250 ml from the Nutrition tab, the Home tab or the home-screen widget without opening the app.

Targets you can check with a calculator



The formula chain

- Basal metabolic rate: Mifflin-St Jeor equation [19]
- Total daily energy: BMR times an activity factor from 1.2 to 1.9
- Calorie target: energy plus or minus a bounded weekly change
- Protein: 2.0, 1.6 or 2.2 g per kg by goal
- Fat: at least 20 percent of calories; carbohydrates take the remainder
- Water: 33 ml per kg, clamped between 1.5 and 4 litres



Floors that cannot be argued down

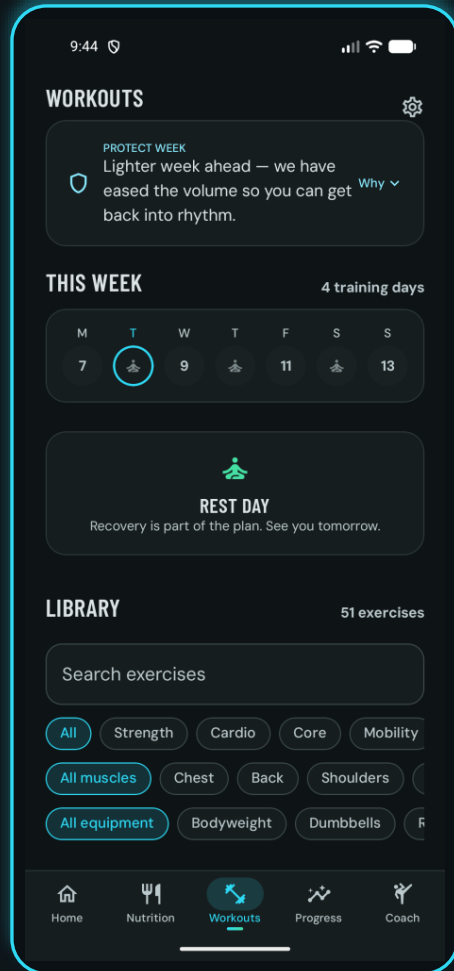
- 1,200 kcal for female and unspecified users, 1,500 kcal for male users, enforced in code and shown as an advisory when they clamp.
- Biometrics outside 30 to 300 kg, 100 to 250 cm or 16 to 100 years are rejected at the boundary, not silently corrected.
- The same constants are shared by the web client and the API from one package, so the platforms cannot drift [3].



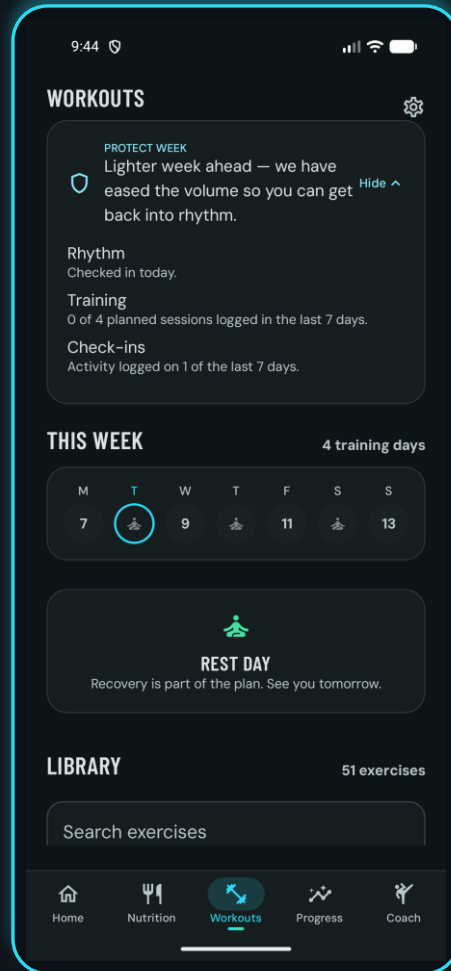
Why this matters

- A number a user cannot reproduce is a number they cannot trust. Every figure on the Nutrition tab is reachable from the profile fields with a hand calculator.
- The Android client re-declares the calculators in pure Kotlin under core/common and tests them on the JVM, with an injected clock so midnight rollover is deterministic [4].
- The Why this target sheet in Settings shows BMR, activity and adaptive adjustments in plain language, which is the explainability most apps hide.
- Mifflin-St Jeor is the equation with the strongest validation for healthy adults, which is why it was chosen over older formulas [19].

Training that adapts to how ready you are



Readiness week



The Why sheet



Guided session



Readiness engine

Rhythm, training load and check-ins set a weekly volume multiplier of 0.6 (protect), 1.0 (maintain) or 1.1 (progress) [5].



Live session service

A foreground service of type health carries the rest countdown and its +30 s and skip actions to the lock screen [16].



Progressive overload

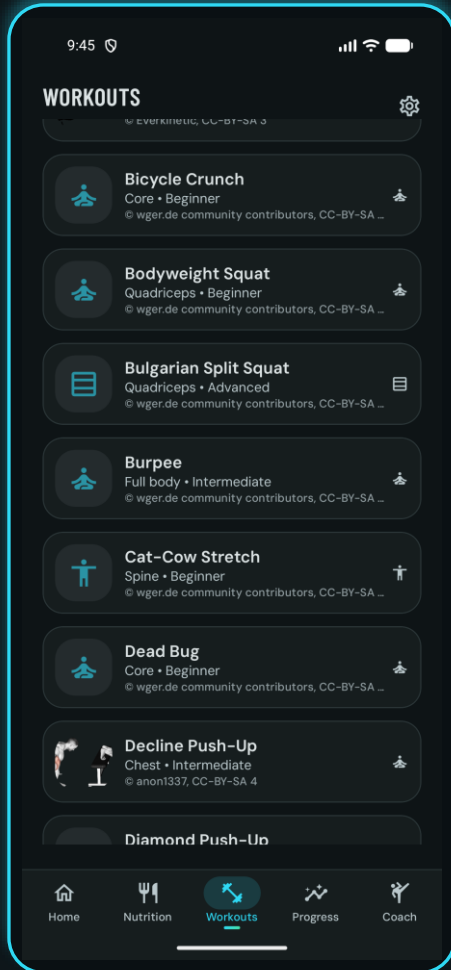
A pure Kotlin engine recommends the next load per movement class: compound lower, compound upper, isolation or bodyweight.



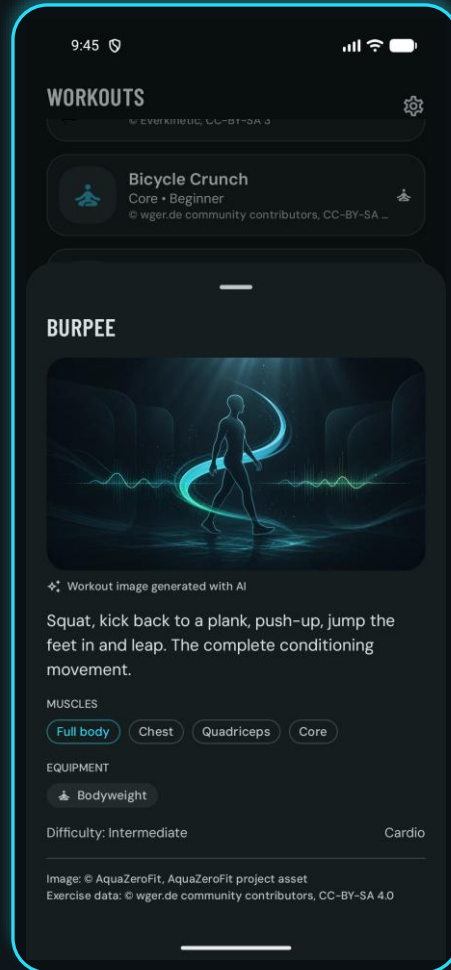
Session burn

Completed sessions credit calories back into the day's derivation row, offline, from the cached session entity.

An exercise library that credits its sources on every card



51 exercises with filters



Detail sheet with credit line



Openly licensed data, handled properly

- Exercise records come from the wger community corpus, licensed per record under CC-BY-SA, CC-BY or CC0 [20].
- The importer copies each record's licence and author verbatim; no pipeline strips them. The card shows the credit under the description and beneath each media item.
- Demonstration images are labelled as AI-generated on the card where that is the case, so users are never misled about provenance.
- The app itself is AGPL-3.0-or-later, the same licence family as the ecosystem it draws from [22].



Filtering before assembly

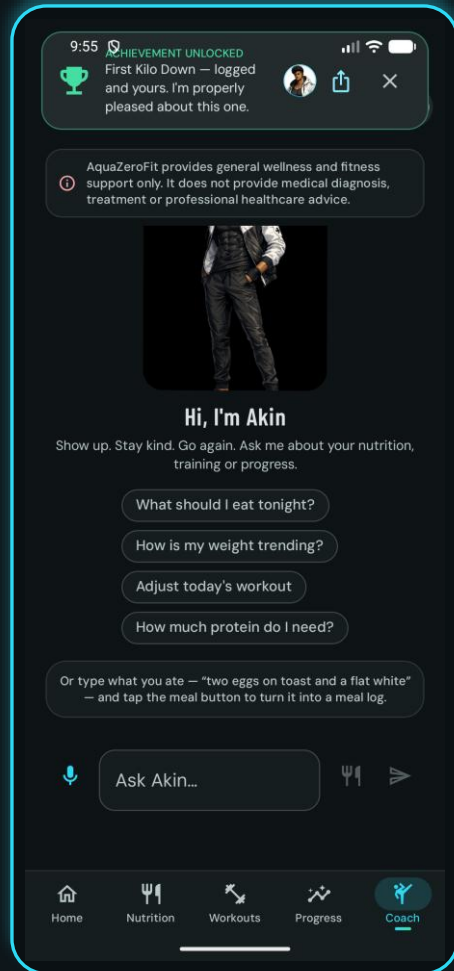
- Equipment, then muscle group, then movement type.
- Profile injuries and equipment filter the library server-side, which is why the endpoint is never cached publicly.



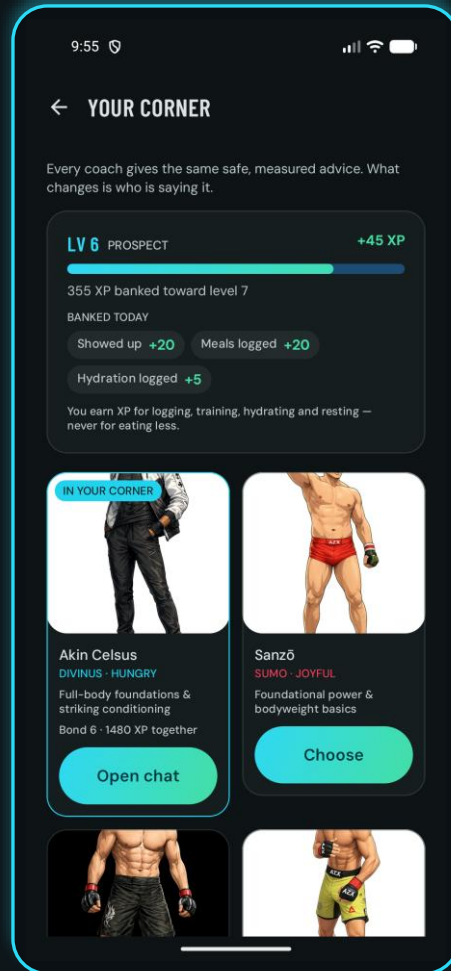
Offline library

- The catalogue is cached in Room, so the library opens with no network.
- Media is mirrored from the app's own origin, never hot-linked from a volunteer-run server.

A coach that knows your day and knows its limits



Akin, boundary pinned



Coach roster and rank



Grounded every turn

- Today's nutrition, the current workout, the progress summary and confirmed memory facts are injected per turn.
- Replies stream over server-sent events, token by token, on a dedicated OkHttp client.
- Voice input through RECORD_AUDIO; a typed meal sentence becomes a draft log.



Bounded in code

- Input and output pass a deterministic classifier first: crisis beats medical beats extreme diet beats out of scope [5].
- A crisis message stops the model and signposts Lifeline 13 11 14 [27].
- Numeric claims are checked against the calorie floor before display.



Personas, memory and consent

- 28 coach identities are defined and 19 ship with character art; Akin is the default. Progression, XP and unlocks are decided by the server, never by the client.
- Coach memory holds facts the user has confirmed, capped at 60, reviewable and deletable in Settings. Personalisation is off until explicit consent and revocable at any time.
- Every AI action costs credits from an append-only ledger: 1 for a message, 3 for a meal photo, 5 for a plan. A degraded (template) answer is never billed [5].

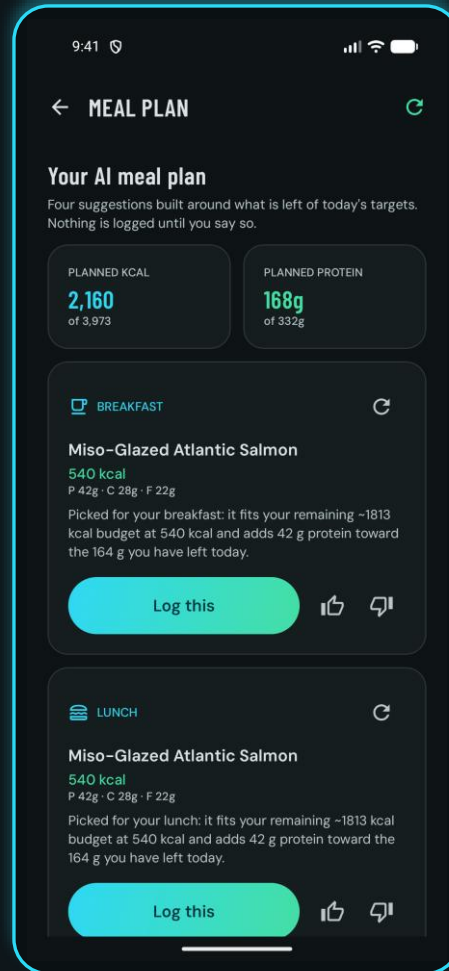
Readable progress and a metabolism model that explains itself



Consistency and weight journey



Calorie trend and badges



Meal suggestions per slot



Adaptive expenditure engine

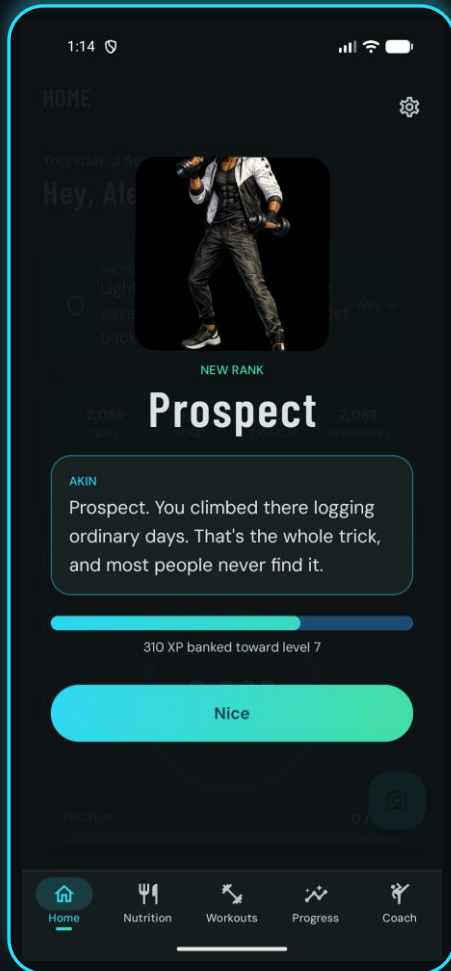
- An exponentially weighted moving average smooths scale weight, then weight velocity is matched against logged intake over a rolling window.
- Adjustments are clamped to plus or minus 100 kcal per week and never breach the safety floors.
- Confidence is reported as high, moderate or low, with a written reason.



Honest insights

- Insights describe the data. They never diagnose and never moralise.
- Meal suggestions are one per slot for today, built around what is left of the targets, and nothing is logged until the user taps Log this.

Motivation that never punishes



Rank-up moment



Bond and level on Home



Experience never decreases

A ratchet over server snapshots clamps any regression on the way in. In a product that also holds health data, taking points away is a harm, so the client refuses to render one. Levels follow a triangular curve; ranks such as Prospect unlock at fixed levels [4].



Consistency, not streaks

- Active days inside a rolling 28-day window with one grace day.
- A best figure that never goes down; badges are judged against it, so an earned badge is never revoked [5].
- A daily experience cap of 150 points keeps the loop about habits, not grinding.



Celebration done right

- Confetti, haptics and the coach's own line run through one CelebrationHost.
- The server is only told a moment was delivered after it has actually been on screen, so a crash never swallows a rank-up.
- Buddy huddles: private challenges of up to four people, shared by deep link.

Why it beats the field

Capability	AquaZeroFit	MyFitnessPal	MacroFactor	Hevy / Strong	Noom
Photo logging with mandatory confirm step	Yes	Partial	Partial	No	Partial
Barcode scanning on the free tier	Yes	No	No free tier	n/a	Yes
Formula shown for every target (Why sheet)	Yes	No	Partial	n/a	No
Adaptive expenditure model	Yes	No	Yes	No	No
Guided sessions with lock-screen rest timer	Yes	No	No	Yes	Partial
AI coach with coded crisis and medical guardrails	Yes	Undocumented	Undocumented	n/a	Undocumented
Offline-first logging with idempotent sync	Yes	Partial	Partial	Yes	No
Per-record data licence shown in the app	Yes	No	No	No	No
Every feature available on the free tier	Yes	No	No	Partial	No
Open source (AGPL-3.0-or-later)	Yes	No	No	No	No

Competitor cells summarise features each vendor documents publicly on its own website and support pages, reviewed in September 2026 [24]. Partial means the capability exists with a material restriction, such as a paid tier or no confirmation step. Undocumented means the vendor does not publish how the behaviour works, which is itself the point: AquaZeroFit publishes its classifier, its formulas and its source.

PART TWO

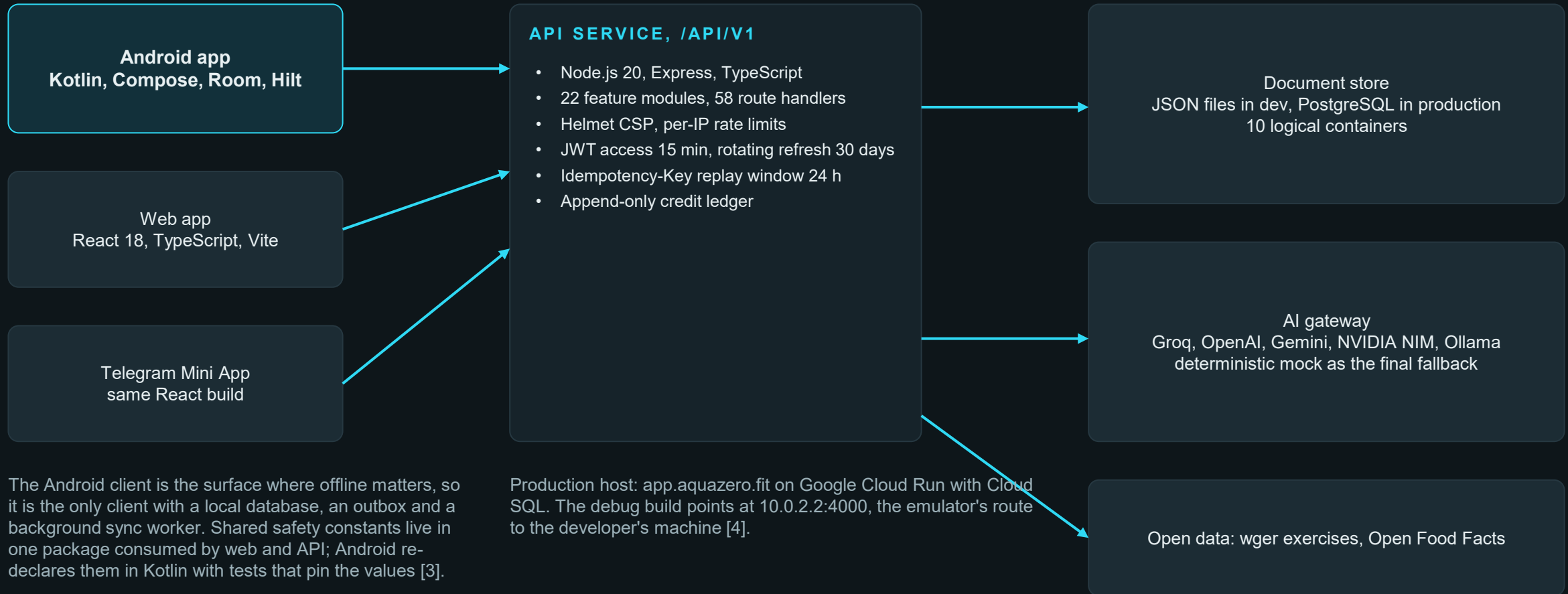
Technical architecture

How the Android client is put together: the stack, the layered package graph, unidirectional state, the offline outbox, networking and secrets, dependency injection, native platform features, the backend it speaks to, and the engineering that gets it onto Google Play.

Numbers on the following slides are measured from the repository, not estimated [3][4].



System context: three clients, one frozen API contract



Android technology stack, pinned versions

Layer	Technology	Version	Role in the app
Language	Kotlin, JVM target 17	2.3.20	All production and test code; coroutines and Flow for async [29]
UI	Jetpack Compose, Material 3, Compose BOM	2026.08.00	Every one of the 21 screens is declarative; no XML layouts [7]
Navigation	Navigation 3	1.1.7	Type-safe back stack; Hilt ViewModel integration without the Nav 2 artifact
Persistence	Room 3 with KSP	3.0.2	23 entities, 9 DAOs, exported schema, migration tests [8]
DI	Hilt (Dagger)	2.60.1	14 modules, 25 HiltViewModels, constructor injection only [9]
Network	Retrofit, OkHttp, kotlinx.serialization	3.0.0 / 5.5.0	Three clients: api, authless refresh, SSE streaming
Background	WorkManager	2.11.2	Outbox replay with constraints and backoff [10]
Camera and ML	CameraX, ML Kit barcode	1.6.2 / 18.3.1	Meal capture and on-device barcode scanning [13][14]
Platform	Health Connect, Glance, Play Billing	1.1.0 / 1.2.0 / 9.1.0	Steps and heart rate in, weight out; widget; premium subscription [11][12][15]
Build	Android Gradle Plugin, Gradle, detekt, ktlint	9.3.2 / 9.5.0	compileSdk 37, targetSdk 36, minSdk 26; R8 minified signed release

minSdk 26 covers Android 8.0 and later; targetSdk 36 satisfies Google Play's current target API requirement. Versions are read from gradle/libs.versions.toml in the repository [4].

One Gradle module, layered by package, one-way graph

app: MainActivity, navigation shell, Glance widget, DI wiring

FEATURE: 21 SCREENS, 25 VIEWMODELS

dashboard

nutrition

training

progress

coach

settings

onboarding

challenges

CORE: SHARED, FEATURE-AGNOSTIC

model

DTOs, ApiResult; no Android imports

common

energy, nutrition, overload, XP maths

database

Room 3, 23 entities, schemas

network

Retrofit, 3 OkHttp clients, SSE

auth

Keystore vault, session lifecycle

sync

outbox, WorkManager, connectivity

designsystem

Compose primitives, motion

ui

shared composables, formatters

gamification

celebration, confetti, brag cards

health

Health Connect gate

vision

pose landmark maths

telemetry

crash and analytics seams



The invariant

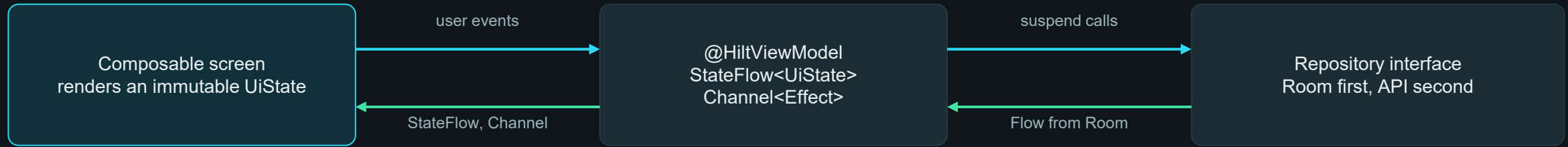
- No feature imports another feature, and nothing under core imports a feature.
- Phase 1 of the modularisation plan removed every feature-to-feature edge; the graph is now measured, not assumed [4].
- Until Gradle modules enforce it, a two-line grep in the README prints nothing when the graph is clean.



Why one module today

- Build files for empty modules would never be validated by Gradle, so they are deliberately not kept on disk ahead of the move.
- The split is specified with the measured dependency graph and execution order in docs/plans; it follows Google's recommended layering [6].

State flows one way: UiState out, events in, effects on a channel



Rules the code keeps

- ViewModels emit string resource ids, never formatted text, so they hold no Context and run on the JVM.
- One-shot effects (toast, navigation, share sheet) go through a Channel so a recomposition cannot replay them.
- An injected `java.time.Clock` makes midnight rollover deterministic in tests.



Testing at the seam

- Hand-written fakes at each @Binds interface: `FakeDashboardData`, `FakeNutritionData`, `FakeCoachesRepository`.
- `StandardTestDispatcher` with `Dispatchers.setMain` for every ViewModel test.
- 518 JVM tests across 63 classes run in CI on every push [4].



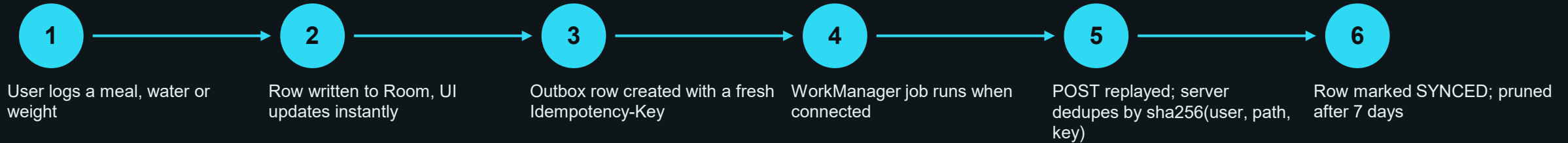
Sample of the shape

```

data class DashboardUiState(
    val kcalTarget: Int,
    val kcalConsumed: Int,
    val readiness: Readiness?,
    val messageRes: Int? = null,
)

sealed interface Effect {
    data class Toast(val res: Int) : Effect
}
  
```

Offline-first: Room first, outbox replay, idempotent sync



Why it cannot double-log

- The key is generated once per operation and never reused with a different payload; the server replays the cached response for 24 hours [4].
- A double-send after a dropped response therefore returns the original result instead of a second row.



Separation for testability

- OutboxDrainer decides what to send and needs no Context, so it is tested on the JVM.
- SyncWorker is only the shell that maps DrainResult onto a WorkManager Result with retry and backoff [10].
- A ConnectivityMonitor triggers a drain the moment the network returns.



What stays online-only

- Coach chat, meal photo analysis and meal suggestions call models, so they require a connection by design.
- Meal photos queue through their own upload worker so a capture in a basement is analysed later, not lost.
- Failures are never pruned; they stay visible until the user resolves them.

Networking and secrets: three clients, one refresh path, one vault

api

Headers plus TokenAuthenticator. Every authenticated call.

authless

Headers only. Used by the refresh coordinator so a refresh can never trigger itself.

sse

The api client with a 180 s read gap for streaming coach replies.



Token lifecycle

- Access token: JWT, 15 minutes, held in memory [25].
- Refresh token: opaque, 30 days, single use, rotated; reuse revokes the family.
- 401 triggers one refresh in flight; callers wait on it rather than racing.



Keystore vault

- Refresh tokens are sealed with AES-GCM using a key generated inside the Android Keystore, StrongBox when available [28].
- A failed StrongBox provisioning falls back rather than crashing the sign-in tap.
- KeystoreAead is an interface so tests substitute a fake.



The deadlock that was designed out

Deriving the clients from one builder copied the Dispatcher reference, so five concurrent 401s filled the per-host budget and the refresh queued behind the calls waiting on it. Each client now owns its Dispatcher; only the connection pool is shared. The comment in NetworkModule records why [4].



Session and isolation

- SessionManager owns sign-in, refresh failure and sign-out; a different user signing in purges every user-scoped table and the outbox before the first read.
- Debug builds talk to the local API; legal pages always open the published site so a privacy policy is never served from a laptop.
- Default cache policy is private, no-store; only byte-identical catalogue reads opt into a five-minute public cache, per RFC 9111 [25].

Dependency injection with Hilt, end to end

14

@Module classes

25

@HiltViewModel classes

73

files with @Inject

0

service locators or static singletons



Interfaces where a seam is needed

- DashboardData, NutritionData, CrashReporter, AnalyticsTracker, KeystoreAead and CoachVoiceEngine are bound with @Binds.
- Tests substitute fakes at exactly those seams; nothing else is mocked.
- @Named qualifiers separate the three OkHttp clients and two Retrofit instances [9].



Hilt in tests too

- AzfTestRunner swaps the Application for instrumented runs.
- FakeNetworkModule replaces the entire network graph, so device tests never touch the real API.
- HiltWorker lets WorkManager workers take constructor dependencies.



The one exception, on purpose

- EntryPointAccessors appears exactly once: in the Glance widget, because a widget receiver cannot be @AndroidEntryPoint [15].
- Everything else is constructor injection, which is what keeps 518 tests on the JVM.

Native superpowers the web client cannot match



CameraX meal capture

In-app capture with framing guidance; gallery import goes through the system Photo Picker, so no storage permission [14].



ML Kit barcode

On-device scanning through a Play Services dynamic module, prefetched when the Nutrition tab opens [13].



Health Connect

Steps, heart rate, sleep and total calories read; weight written back. Nothing is read until the user connects, re-checked on every call [11].



Glance widget

Today's rings and a one-tap water log drawn from Room, so it is correct with no network [15].



Foreground live workout

A health-type foreground service carries the rest countdown and its actions to the lock screen [16].



Google Play Billing

Server-verified subscriptions through purchases.subscriptionsv2; renewals self-heal and the account id is obfuscated [12].



Coach voice

RECORD_AUDIO for spoken questions to the coach, behind a swappable CoachVoiceEngine interface.



On-device form check

Pose landmark maths for squat depth runs locally; no frame ever leaves the phone.

Each capability sits behind an interface or a repository so a screen degrades to an honest empty state when the capability is absent, rather than crashing [4].

The backend the app speaks to



Modules

- auth, me, foods, logs, analytics, plans, workouts, progress, recipes, admin, chat, vision, recommendations, challenges, coaches, payments, billing, export, mfa, memory, ai.
- 22 module directories, 58 route handlers, all validated at the boundary with shared zod schemas [3].



Storage abstraction

- One document-store interface with two backings: JSON files for development and tests, PostgreSQL for production.
- Ten logical containers; user-scoped ones are what export and purge iterate.
- Single-instance durability by design; scale-out requires moving reads off the local working set, and that limit is recorded, not hidden [5].



Operational posture

- Health and readiness probes outside the rate limiter.
- Request-id correlation through the stack and persisted error records.
- Graceful shutdown with a 25 second drain budget; background sweeps for photo TTL and account deletion grace.
- Production refuses to boot on dev secrets [3].

AI gateway and safety pipeline

visionPrimary

chatFast

planStructured

safetyCheap

insightBatch

Application code names only these logical lanes; providers, timeouts and fallbacks live in one file.

Groq

OpenAI

Gemini

NVIDIA NIM

Ollama

Deterministic
mock

Ordered fallback chain: retries with jittered backoff, a per-provider circuit breaker, one overall deadline, and a degraded flag when only the mock answered.



What the model may and may not do

- **May:** identify foods, propose quantities, write a coaching sentence, summarise a week, suggest a swap.
- **May not:** write a calorie figure, override a floor, choose an allergen exclusion, decide an entitlement, or answer a crisis message.
- Every call, real or mock, is logged with its lane, provider and outcome.
- Prompts are versioned files P-01 to P-12 at the repository root; changing one is a reviewed commit [3].
- Consent gates: personalisation and memory extraction never run for an account that has not opted in.



Two-stage classifier

- Stage one is deterministic pattern matching in code on both input and output.
- Priority: crisis, then medical, then extreme diet, then out of scope [5].
- 498 evaluation cases gate the pipeline: 50 assistant, 319 plan, 129 recommendation.



Credit ledger

- Append-only; the balance is a fold over transactions, never a mutable counter.
- Reserve before work, commit after, release on failure, so a crashed job returns its hold.
- Free tier 10 credits a day, premium 150, banked to a cap.

Security, privacy and a deliberately lean permission set

CAMERA

meal and barcode capture

POST_NOTIFICATIONS

reminders, live session

FOREGROUND_SERVICE_HEALTH

live workout timer

health.READ_* (4)

steps, heart rate, sleep, calories

INTERNET

API access

RECORD_AUDIO

coach voice

ACTIVITY_RECOGNITION

prerequisite for the health service

health.WRITE_WEIGHT

weigh-ins back to Health Connect



Data handling

- Meal photos: size-capped, MIME allow-listed, EXIF and location stripped, unguessable names, streamed only to the owner, deleted on confirm and swept after 24 hours.
- Export my data as JSON and a 30-day account deletion grace period are in Settings, matching the Australian Privacy Principles on access and correction [26].
- Consents are granular and revocable: wellness processing, AI personalisation, anonymised analytics, reminders.



Not requested, on purpose

- No READ_MEDIA_*: gallery import uses the system Photo Picker.
- No location, no storage, no exact alarms, no QUERY_ALL_PACKAGES, and the advertising id permission is stripped from the merged manifest and verified in the release report.
- Every permission is a Play review surface, so each one has to earn its place [23].

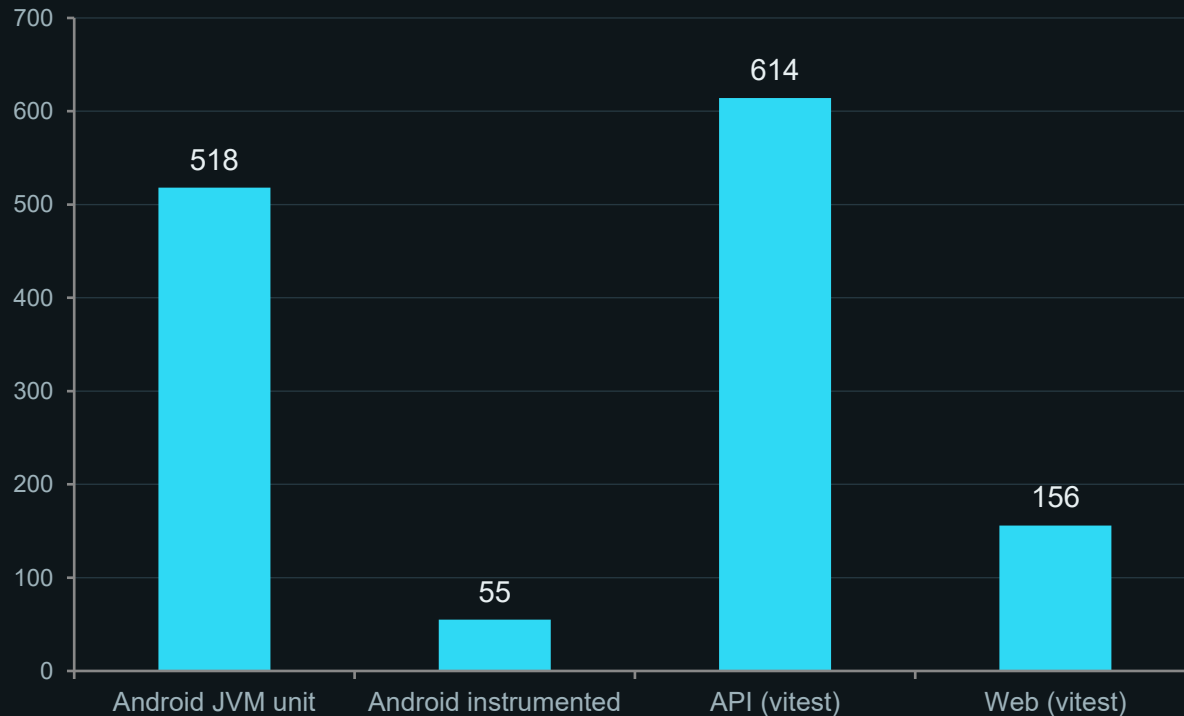


Defence in depth

- Rate limiting per IP with a separate authentication lane; bcrypt password hashing; HS256 JWTs with short life; rotating refresh tokens with family revocation.
- Client secrets sealed in the Android Keystore; the FileProvider for export is not exported and grants a read-only URI for one file.
- R8 minification on release; the release build refuses to package unsigned [4][17].

Quality: 1,343 automated tests, four gates on every push

Passing tests by suite (September 2026)



Android counts from the README at the cited commit; platform counts from the presentation guide [4][5]. These are test counts, not a coverage percentage; line coverage has not been measured.



ktlintCheck

formatting and import order



detekt

complexity and smells, every raised threshold has a recorded reason



lintDebug

Android Lint: 0 errors, 156 warnings each classified in the README



testDebugUnitTest

518 JVM tests in 63 classes

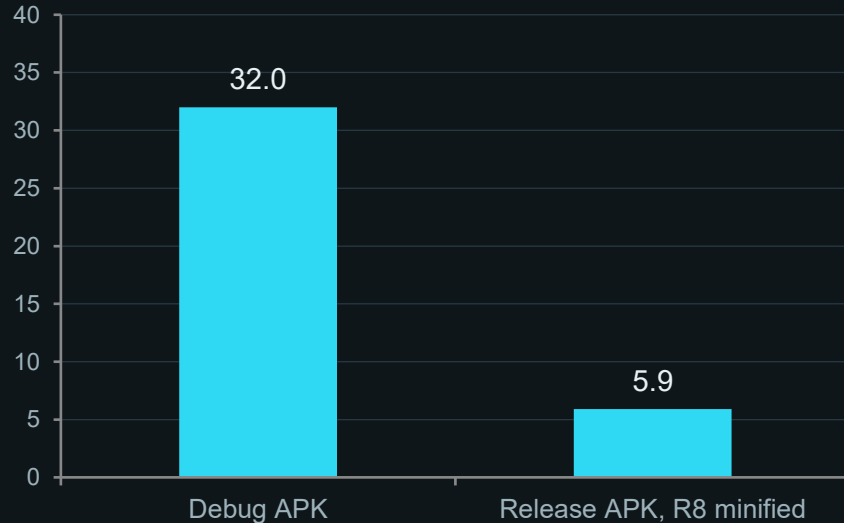


What the device tests cover

Room DAOs and the outbox state machine, schema-migration readiness against the exported schema, navigation through the signed-in and pre-auth shells, the workout library screen, and purge-on-user-switch. The README warns to trust the test count over the green banner, because a poisoned build cache once ran a truncated suite [4].

Release engineering: a build that refuses to ship wrong

82 percent smaller after R8 and resource shrinking



Measured locally on 29 August 2026; the minified build launched clean with Hilt, Room and serialization intact [17].

Release builds point at <https://app.aquazero.fit>; debug builds point at the developer machine. The full procedure is in [docs/PLAY_STORE_RELEASE_GUIDE.md](#) [3].



Signing guard

A missing keystore used to degrade silently to an unsigned APK with a green build. It is now a hard failure at packaging time; the only escape hatch is an explicit local flag CI never passes.



Version from the tag

versionCode and versionName come from the release tag. Play rejects a re-used versionCode, so a constant would have failed every upload after the first.



Two workflows

android-verify runs the four gates on every push touching apps/android and references no secrets. android-release runs on a tag: gates, signed APK and AAB, then apksigner verification.



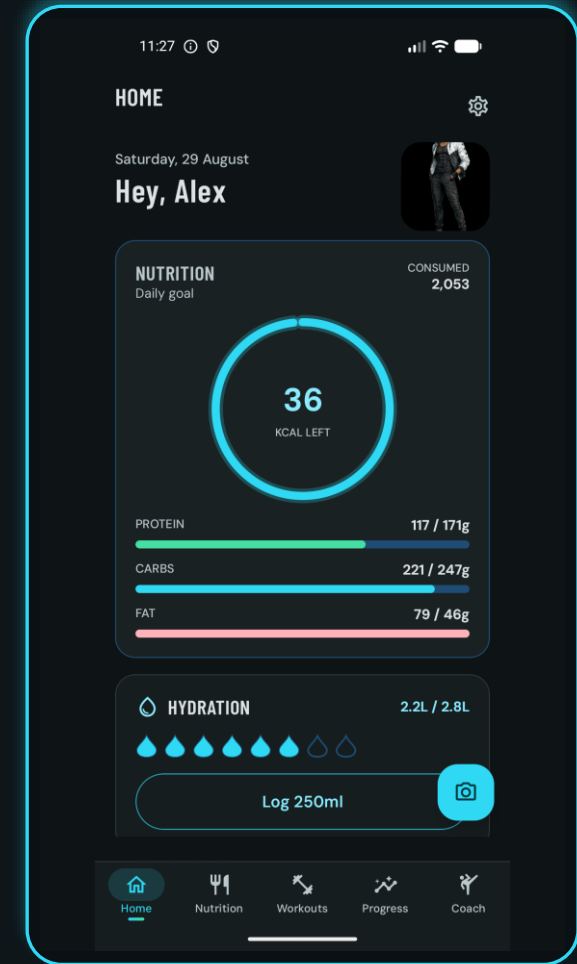
Static analysis

detekt pinned to a 2.0 pre-release because the last 1.x cannot run on the JDK 25 shipped with Android Studio; the reason is recorded beside the pin.

Google Play readiness

Console task	Status	Basis
App identity	fit.aquazero.app, Luminara Digital Pty Ltd	Matches applicationId and launcher label
Category and health	Health and fitness; activity and nutrition declared	Chosen over Medical; the app disclaims medical advice in 11 places
Data safety	Submitted, 14 data types	Encrypted in transit; deletion via the support page
Content rating	Rated 3+, in-app purchases declared	IARC questionnaire; AI content answered as online content
Target audience	16 to 17 and 18 and over	Matches the Terms and the minimum age constant
Advertising id	Not used, verified in the signed binary	Stripped in the manifest merge report
Store listing	Name, 76-character short description, 3,687-character full description, icon, feature graphic, 7 screenshots	Every number traceable to the shared constants
Remaining	Upload AAB, closed testing with 20 testers for 14 days, Health Connect declaration if in scope	Play policy for new developer accounts [23]

Screenshot order is deliberate: the first answers what the app is; the scan and the confirm step sit side by side; the fourth is the trust close [3].



Play listing screenshot 1 of 7, padded to 9:16 in the app's own surface colour

How it was built: Android Studio, Gradle and agent tooling



Specify first

A document set from charter to test plan (AQF-01 to AQF-29) fixed the API contract, the data model and the safety invariants before the first Kotlin file.



Build in Android Studio

Kotlin 2.3 with the Compose compiler plugin, KSP for Room and Hilt, a version catalogue, and the bundled JBR as the JDK on Windows.



Run on the emulator

A Pixel virtual device against the local API on 10.0.2.2; the debug build is installed with installDebug or the Android CLI [18].



Verify the same way CI does

ktlint, detekt, lint and the JVM suite locally, then instrumented tests on the device, then a minified release smoke test.



Agent tooling

The Android CLI for agents was used to describe the project, drive the emulator and capture the screens in this deck, alongside code review by an AI assistant with a human in charge of every decision [18].



Write it down

Every non-obvious decision is a comment in the code or a line in a README, so the next engineer inherits the reason, not just the rule.

Development diary, progress report and final report are in docs/specs; the Android implementation plan and modularisation plan are in docs [3].

Evidence against the NIT3213 learning outcomes

Learning outcome	Where this project demonstrates it
1. Design and develop mobile applications using current software technologies	A complete native Android application in Kotlin 2.3 and Jetpack Compose with Room, Hilt, WorkManager, CameraX, ML Kit, Health Connect and Play Billing, built, tested and packaged for Google Play (slides 17 to 28).
2. Create graphical user interfaces that follow design principles and guidelines	Material 3 components, a documented design system (Modern Aquatic Wellness), consistent card tiers and motion, dark-surface contrast, unidirectional state, and Compose screens captured live on a Pixel emulator (slides 5 to 13).
3. Explain core concepts underlying contemporary mobile platforms	Activity lifecycle and process death handled through ViewModel state, foreground services and their permission gates, background work with WorkManager constraints, the Keystore, permissions as review surfaces, and R8 shrinking (slides 18 to 26).
4. Apply ethical considerations relevant to mobile application development	Confirm-first logging, calorie floors enforced in code, crisis signposting, consent-gated personalisation, metadata stripping, a lean permission set, per-record data attribution and an AGPL licence that returns to the commons (slides 4, 11, 25, 26).

The unit description on the Victoria University site lists these four outcomes for NIT3213 Mobile Application Development [1].

Known limits, stated plainly, and what comes next



Limits we record rather than hide

- Single Gradle module: layering is enforced by convention and a grep until the planned module split lands.
- The API scales as a single instance because each instance serves reads from its own in-memory working set; scale-out needs reads moved off the local copy [5].
- Lint reports 156 warnings; 36 are plural-string candidates that must be fixed before any locale beyond English.
- Test counts are not coverage; line coverage has not been measured.
- Health Connect ships behind a Play declaration that may be deferred from version 1.
- Closed testing with 20 testers for 14 days has not started; the store is prepared, not live.



Roadmap

- Phase 2 and 3 of modularisation: per-layer Gradle modules with the compiler enforcing the graph.
- Closed testing track, staged rollout from 10 to 100 percent.
- Portion memory: remember a user's corrections to portion sizes.
- Squat form check from the on-device pose engine, squat only at first.
- Health Connect signals feeding readiness, and a protein-floor mode for users on appetite-suppressing medication, with clinician sign-off on the copy.
- Localisation once the plural-string work is done.

Conclusion

AquaZeroFit for Android is a native, offline-first Kotlin client that lets a language model do only what it is good at, identifying and explaining, while deterministic code owns every number, every floor and every safety decision. It is built on the current Android toolchain, verified by 573 Android tests and four CI gates, packaged as a signed 5.9 MB release, and prepared for Google Play with a lean permission set and an honest data-safety declaration.



Trustworthy

Targets you can check, confirm-first logging, attribution on every record.



Resilient

Room as the source of truth, idempotent sync, AI that degrades honestly.



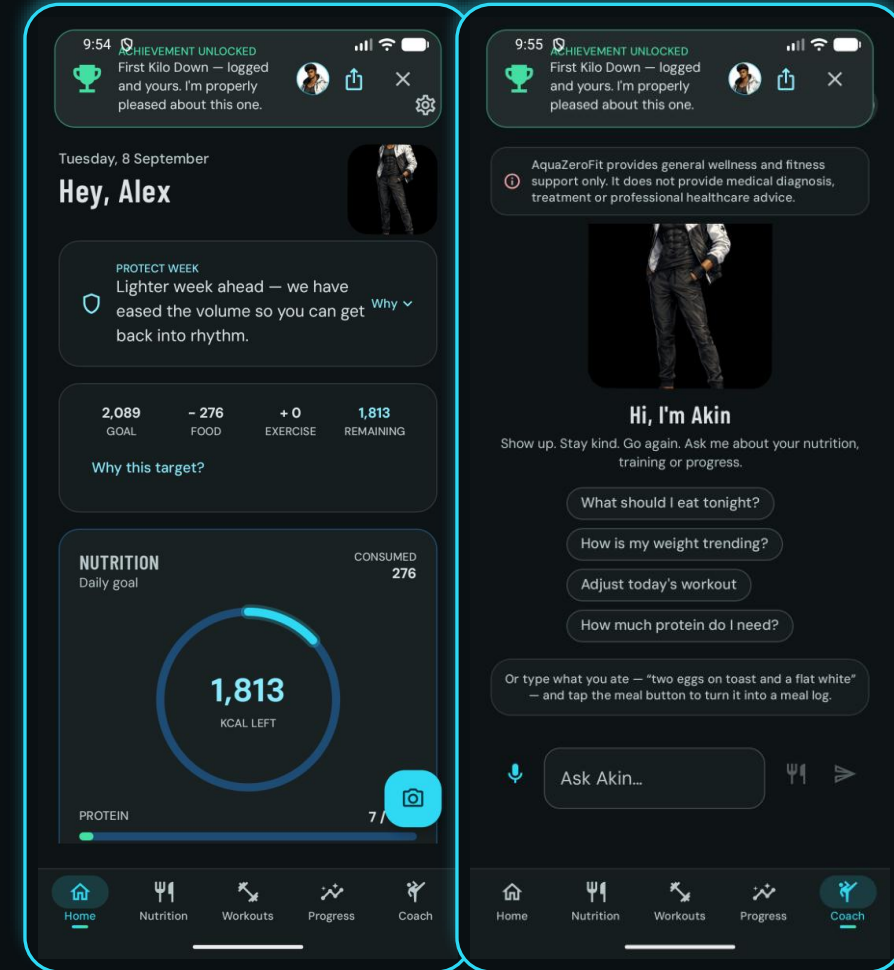
Safe by construction

Floors, allergens and crisis handling in code, consent before personalisation.



Open

AGPL-3.0-or-later, open data credited, decisions written down.



Thank you. Questions are welcome; the code is public [3].

References (1 of 2)

- [1] Victoria University. (2026). NIT3213 Mobile Application Development, unit description and learning outcomes. <https://www.vu.edu.au/units/mobile-application-development-nit3213>
- [2] AquaZero. (2026). AquaZeroFit official website. <https://aquazero.fit.com/>
- [3] Luminara Digital. (2026). aquazero.fit: source repository, documentation set AQF-01 to AQF-29, Play Store guides and plans. <https://github.com/LuminaraDigital/aquazero.fit>
- [4] Williams-Fulwood, B. (2026). AquaZeroFit for Android, README and source (apps/android), repository commit 17faa2d, 8 September 2026.
- [5] Williams-Fulwood, B. (2026). AquaZeroFit Architecture and Presentation Guide, and Architectural and Technical Report (docs/plans, docs/ARCHITECTURE.md), Victoria University.
- [6] Android Developers. (2026). Guide to app architecture. <https://developer.android.com/topic/architecture>
- [7] Android Developers. (2026). Jetpack Compose. <https://developer.android.com/compose>
- [8] Android Developers. (2026). Save data in a local database using Room. <https://developer.android.com/training/data-storage/room>
- [9] Android Developers. (2026). Dependency injection with Hilt. <https://developer.android.com/training/dependency-injection/hilt-android>
- [10] Android Developers. (2026). Schedule tasks with WorkManager. <https://developer.android.com/topic/libraries/architecture/workmanager>
- [11] Android Developers. (2026). Health Connect. <https://developer.android.com/health-and-fitness/guides/health-connect>
- [12] Android Developers. (2026). Google Play Billing Library. <https://developer.android.com/google/play/billing>
- [13] Google. (2026). ML Kit: Scan barcodes. <https://developers.google.com/ml-kit/vision/barcode-scanning>
- [14] Android Developers. (2026). CameraX overview. <https://developer.android.com/media/camera/camerax>
- [15] Android Developers. (2026). Build app widgets with Glance. <https://developer.android.com/develop/ui/compose/glance>

References (2 of 2)

- [16] Android Developers. (2026). Foreground services and foreground service types. <https://developer.android.com/develop/background-work/services/fgs>
- [17] Android Developers. (2026). Shrink, obfuscate, and optimize your app (R8). <https://developer.android.com/build/shrink-code>
- [18] Android Developers. (2026). Android CLI for agents. <https://developer.android.com/tools/agents/android-cli>
- [19] Mifflin, M. D., St Jeor, S. T., Hill, L. A., Scott, B. J., Daugherty, S. A., and Koh, Y. O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241 to 247. <https://doi.org/10.1093/ajcn/51.2.241>
- [20] wger Workout Manager. (2026). Open exercise database and REST API, licensed per record under Creative Commons. <https://wger.de> and <https://github.com/wger-project/wger>
- [21] Open Food Facts. (2026). Open food products database, Open Database License 1.0. <https://world.openfoodfacts.org>
- [22] Free Software Foundation. (2007). GNU Affero General Public License, version 3. <https://www.gnu.org/licenses/agpl-3.0.html>
- [23] Google Play Console Help. (2026). Health apps policy, data safety section, and app testing requirements for new developer accounts. <https://support.google.com/googleplay/android-developer>
- [24] MyFitnessPal (<https://www.myfitnesspal.com>), MacroFactor (<https://macrofactorapp.com>), Hevy (<https://www.hevyapp.com>), Strong (<https://www.strong.app>) and Noom (<https://www.noom.com>). Product feature pages and support documentation, accessed September 2026.
- [25] Jones, M., Bradley, J., and Sakimura, N. (2015). RFC 7519: JSON Web Token. IETF. Fielding, R., Nottingham, M., and Reschke, J. (2022). RFC 9111: HTTP Caching. IETF. <https://www.rfc-editor.org>
- [26] Office of the Australian Information Commissioner. (2026). Australian Privacy Principles. <https://www.oaic.gov.au/privacy/australian-privacy-principles>
- [27] Lifeline Australia. (2026). Crisis support, 13 11 14. <https://www.lifeline.org.au>
- [28] Android Developers. (2026). Android Keystore system. <https://developer.android.com/privacy-and-security/keystore>
- [29] Kotlin Foundation. (2026). Kotlin programming language documentation. <https://kotlinlang.org/docs/home.html>

Glossary for non-technical readers

Term	Plain meaning
Native app	Software written for Android itself, in Kotlin, rather than a website shown inside an app shell.
Jetpack Compose	Google's current way of drawing Android screens from code, replacing XML layouts.
Room	The on-phone database. Here it is the source of truth, so the app works without a network.
Outbox and idempotency key	A queue of unsent changes, each tagged with a one-time code so a retry can never create a duplicate.
Hilt	The wiring that hands each part of the app the pieces it depends on, so parts can be swapped in tests.
Foreground service	A background task Android lets keep running with a visible notification, used for the live workout timer.
Health Connect	Android's shared health data store, used only after the user connects it and re-checked on every read.
R8	The build step that removes unused code and shrinks the app from 32 MB to 5.9 MB.
Mifflin-St Jeor	The published equation used to estimate resting energy needs from weight, height, age and sex.
AGPL-3.0-or-later	An open-source licence: anyone who runs a modified version as a service must publish their changes.